



Machine Learning–Based Late Delivery Risk Prediction in Global Supply Chain Operations

Domain: Supply Chain Analytics & Logistics

Dataset: APL_Logistics.csv · 180,519 records · 40 features

Target: Late_delivery_risk (Binary Classification)

Best Model: XGBoost Classifier · ROC-AUC: 0.8146

Tools: Python · scikit-learn · XGBoost · SHAP · Streamlit

Year: 2025

Prepared by: Analytics & Data Science Team

Table of Contents

1. Background & Problem Statement	3
2. Dataset Overview	4
3. Exploratory Data Analysis (EDA)	5
3.1. Target Variable Distribution	5
3.2. Shipping & Market Analysis	5
3.3. Regional & Order Status Analysis	6
3.4. Correlation Analysis	6
4. Data Preprocessing	7
5. Feature Engineering	8
6. Encode, Split & Balance	9
7. Model Development	10
7.1. Logistic Regression (Baseline)	10
7.2. Random Forest Classifier	10
7.3. XGBoost Classifier	11
8. Cross-Validation & Model Selection	12
9. Risk Probability & Classification	13
10. SHAP Explainability	14
11. Artifacts & Streamlit Deployment	15
12. Conclusion & Recommendations	16

1. Background & Problem Statement

Why this project exists

In global logistics networks, late deliveries are a critical operational challenge. They trigger SLA breaches, penalty clauses, customer churn, and significant last-minute recovery costs. Traditional approaches handle delays *reactively* — after the failure has already occurred — leaving operations teams without the foresight needed to act proactively.

The Core Business Problem

APL Logistics faced a specific three-pronged challenge that this project was designed to address:

- **Unpredictable shipment delays** — no early warning system to identify at-risk orders before they are shipped.
- **High operational cost of late fixes** — last-minute rerouting, emergency shipping upgrades, and manual exception handling are expensive.
- **Limited prioritisation capability** — without quantitative risk scores, operations teams cannot systematically prioritise high-risk orders in their daily queues.

Project Mission: Build a machine learning system that predicts, before an order is shipped, the probability of a late delivery — and surfaces that intelligence through an interactive dashboard enabling proactive risk management.

What Makes This Different from Historical Analysis

Earlier analytical work on APL Logistics focused on understanding *why* delays happened in the past — a retrospective lens. This project shifts to a *prospective* lens: which orders are likely to be delayed before they leave the warehouse. This distinction is operationally critical — only forward-looking predictions allow teams to intervene in time.

Key Deliverables

Deliverable	Description
Trained ML Models	Logistic Regression, Random Forest, XGBoost with serialised artifacts
Risk-Scored Dataset	All 180K orders enriched with probability score and risk category
SHAP Explainability	Feature importance and directional impact plots for model transparency
Streamlit Dashboard	4-tab interactive app for risk overview, prediction, and action planning

2. Dataset Overview

APL_Logistics.csv — 180,519 rows × 40 columns

180,519 Total Records orders	40 Total Features original columns	Binary Target Variable Late_delivery_risk (0/1)	latin1 File Encoding non-UTF-8 characters present
---	---	--	--

The dataset captures a complete picture of global order fulfilment across five major market regions (LATAM, Europe, Pacific Asia, USCA, Africa) and spans customer demographics, order financials, product details, and logistics metadata. Below is a categorised summary of the 40 original fields.

Feature Categories

Category	Columns	Purpose
Customer Info	Customer City, Country, State, Segment, Fname, Lname, Id	Customer demographics & identity
Order Financials	Sales, Benefit per order, Order Item Total, Profit Ratio, Discount	Revenue, margin, and discount data
Logistics	Days for shipping (real), Days for shipment (scheduled), Shipping Mode, Delivery Status	Core delivery time and mode data
Geography	Market, Order Region, Order City, Order Country, Latitude, Longitude	Spatial delivery context
Product	Product Name, Product Price, Category Name, Department Name	What was ordered
Order Meta	Type (payment), Order Status, Order Customer Id	Payment method and order state
Target	Late_delivery_risk	1 = Late, 0 = On Time (binary)

Data Quality Notes

- **Missing values** were minimal: only **Customer Lname** (8 rows) and **Customer Zipcode** (3 rows) had nulls — both handled during preprocessing.
- **Encoding:** The file uses **latin1** (ISO-8859-1) encoding due to special characters in location names. Loading with UTF-8 raises a UnicodeDecodeError on byte 0xe9.
- **No duplicate rows** were detected after initial inspection.
- **Scale variance:** Numerical features span very different ranges (e.g., Latitude −90 to 90 vs. Order Item Total up to \$4,000), necessitating StandardScaler for distance-based models.

3. Exploratory Data Analysis (EDA)

Understanding distributions, patterns, and relationships

EDA was conducted in five focused sub-analyses. The goal was not just description, but hypothesis generation — identifying which features carry signal for predicting late delivery and flagging data quality issues before modelling.

3.1 Target Variable Distribution

The first step in any classification project is understanding the balance between classes. An extreme imbalance (e.g., 95/5) would fundamentally change modelling choices.

81,542 On Time (0) 45.17% of orders	98,977 Late (1) 54.83% of orders	1.214 Imbalance Ratio Mild — 1.2× more late deliveries
--	---	---

Why this matters: With a ~55/45 split, class imbalance is mild but present. A naive model predicting "always Late" achieves 54.8% accuracy trivially. We addressed this with SMOTE oversampling on training data and `class_weight='balanced'` in applicable models.

3.2 Shipping & Market Analysis

We analysed late delivery rates across key logistics dimensions to understand where risk concentrates.

Shipping Mode	Late Delivery Rate	Insight
First Class	Highest	Tighter SLAs → more pressure → higher failure rate
Second Class	High	Moderate SLA pressure
Standard Class	Moderate	Longer windows reduce risk despite lower priority
Same Day	Lowest	Highest priority — dedicated handling lanes

Counter-intuitive finding: First Class, despite being a premium tier, had the highest late delivery rate. This appears to be because premium modes carry tighter delivery windows — when delays do occur, they more often breach the promised SLA threshold, registering as "late".

Market Region	Order Volume	Risk Profile
LATAM	51,594 (28.6%)	High risk — infrastructure challenges
Europe	50,252 (27.8%)	Moderate risk — well-developed logistics
Pacific Asia	41,260 (22.9%)	Moderate-high risk — long haul routes
USCA	25,799 (14.3%)	Lower risk — domestic infrastructure

Market Region	Order Volume	Risk Profile
Africa	11,614 (6.4%)	Highest per-order risk — limited coverage

3.3 Regional & Order Status Analysis

Regional breakdown at the Order Region level (more granular than Market) revealed that certain geographic corridors consistently carry higher delay probability — primarily cross-continental routes and regions with infrastructure constraints. At the Order Status level:

Order Status	Late Delivery Rate	Interpretation
PENDING_PAYMENT	Very High	Payment delays create fulfilment bottlenecks
PENDING	Very High	Orders not yet confirmed — scheduling uncertainty
PROCESSING	Moderate	In-progress — some risk remains
COMPLETE	Moderate	Completed — historical benchmark
ON_HOLD	Elevated	Explicitly paused — risk of expiry
SUSPECTED_FRAUD	Highest	Fraud review causes extended holds
CANCELED	Elevated	Cancellations often precede late flagging

3.4 Correlation Analysis

A lower-triangular correlation heatmap of all numerical features was computed to quantify linear relationships with the target.

Feature	Correlation with Target	Action Taken
Days for shipping (real)	+0.40 (strong)	DROPPED — data leakage
Days for shipment (scheduled)	−0.37 (strong)	Kept — available pre-shipment
Order Item Profit Ratio	Near zero	Kept — indirect signal
Order Item Quantity	Near zero	Kept — used in FE
Benefit per order	Near zero	Kept — financial context

Key Leakage Discovery: Days for shipping (real) has a +0.40 correlation with the target because it IS the target in a different form — a real shipping day exceeding the scheduled days directly causes a "late" classification. Using it as a feature would be data leakage — the model would learn to cheat rather than generalise.

4. Data Preprocessing

Cleaning, de-leaking, and preparing the data for modelling

Preprocessing translates raw data into a clean, model-ready format. Three distinct categories of columns were removed, and minimal missing values were handled.

Column Removal Strategy

Category	Columns Removed	Reason
Data Leakage	Delivery Status Days for shipping (real)	Delivery Status directly encodes the target. Real shipping days are post-delivery information unavailable before shipment.
PII / Non-predictive	Customer Fname, Lname, Street, Zipcode Customer Id, Order Customer Id	Personally identifiable data with no predictive value for delivery timing.
Redundant IDs	Category Id, Department Id	These are surrogate keys for columns already present (Category Name, Department Name).

Post-drop shape: 180,519 rows × 30 columns — a reduction of 10 columns from the original 40.

Why These Drops Are Critical

The most important removal is **Delivery Status**. This column contains values like "Late delivery", "Advance shipping", "Shipping on time", and "Shipping canceled" — categories that are a direct textual encoding of the target variable `Late_delivery_risk`. Including it would give the model a perfect input feature that it would immediately learn to rely on, producing artificially high accuracy on training data but zero real-world utility (because this status is not known before shipment).

Similarly, **Days for shipping (real)** represents the actual number of days taken — a quantity only measurable after delivery. The key insight from the correlation analysis is that this feature has a 0.40 correlation with the target precisely because it IS the target in disguise.

Missing Value Handling

Column	Missing Count	Strategy	Why
Customer Lname	8	Fill with "Unknown"	Negligible; PII-adjacent
Customer Zipcode	3	Fill with median	Numeric; near-zero impact

With only 11 missing values across 180K rows (0.006%), imputation strategy has negligible impact on model performance. The conservative approach (mode/median fill) was chosen over row deletion to preserve every training sample.

5. Feature Engineering

Creating domain-driven predictive signals

Feature engineering is the process of using domain knowledge to transform existing raw fields into new representations that are more informative for the model. Ten new features were created, organised into four conceptual groups.

Group 1: Schedule Pressure Features

shipping_pressure

Definition: $\text{Order Item Quantity} \div (\text{Scheduled Days} + 1)$

Why: Captures how many items must move per day of available shipping time. High pressure = many items, few days = more likely to miss the deadline.

tight_schedule_flag

Definition: 1 if Days for shipment (scheduled) ≤ 2 , else 0

Why: Binary flag for extremely tight windows. SHAP analysis later confirmed this as the single most predictive feature in the entire model.

Group 2: Shipping Mode Risk Features

express_shipping_flag

Definition: 1 if Shipping Mode is First Class or Same Day, else 0

Why: Separates express from economy tiers. While First Class showed higher SLA breach rates, Same Day has dedicated handling — the binary captures this nuance when combined with other features.

shipping_mode_risk_score

Definition: Ordinal: Standard Class=3, Second Class=2, First Class=1, Same Day=0

Why: Encodes the relative operational risk of each mode as an ordinal variable (3=highest risk), allowing tree models to split on mode risk without one-hot encoding overhead.

Group 3: Regional Congestion Indicators

region_risk_score

Definition: Historical mean of Late_delivery_risk, grouped by Order Region

Why: Target-encodes each region with its empirical delay rate. Regions with chronically poor performance get a high score — this is computed on the full training set and mapped to all rows.

market_risk_score

Definition: Historical mean of Late_delivery_risk, grouped by Market

Why: Same principle at the coarser Market level (LATAM, Europe, etc.), providing a macro-geographic signal complementing the regional score.

Group 4: Order Complexity Features

Feature	Definition	Rationale
order_complexity	Product Price × Order Item Quantity	High-value, high-volume orders require more complex fulfillment
high_value_order	1 if Order Item Total > 75th percentile, else 0	Flags premium orders that may have stricter SLA expectations
high_discount_flag	1 if Discount Rate > 15%, else 0	Deep discounts may signal clearance / non-priority stock with delayed handling
profit_margin	Benefit per order ÷ (Sales per customer + 1)	Low-margin orders may receive lower fulfillment priority

Engineering Impact: The single most important feature in the final XGBoost model — as confirmed by SHAP — was tight_schedule_flag, an engineered binary. None of the original raw features ranked #1 individually. This demonstrates the value of domain-informed feature construction.

6. Encode, Split & Balance

Preparing the feature matrix for model training

Categorical Encoding

All categorical columns were encoded using **Label Encoding** (`sklearn.preprocessing.LabelEncoder`). For each column, the encoder maps each unique string to an integer index. The encoder objects were serialised to disk (`joblib`) so the Streamlit app can apply identical transformations to new user-input orders at prediction time.

Why Label Encoding over One-Hot Encoding? With tree-based models (Random Forest, XGBoost) — which represent the majority of our model set — label encoding is appropriate because tree splits handle ordinal integer representations natively. One-hot encoding on high-cardinality columns like Order Region (20+ values) would create hundreds of sparse binary columns, inflating memory and training time with no benefit for tree models.

Train / Test Split

144,415 Training Set rows (80%)	36,104 Test Set rows (20%)	Stratified Strategy stratify=y preserves class ratio	42 Random State reproducible splits
--	---	--	--

The 80/20 split is standard for datasets of this size. Crucially, `stratify=y` was used — ensuring both the training and test sets maintain the original 55/45 class ratio rather than getting a random split that might skew one set toward a particular class.

SMOTE — Synthetic Minority Oversampling

Despite the mild imbalance (1.214 ratio), SMOTE was applied to the training set only as a defensive measure. SMOTE generates synthetic samples for the minority class (On Time, 0) by interpolating between existing samples in feature space — not simply duplicating rows.

- **Applied to training data only** — the test set is never touched, preserving its role as an unbiased evaluation benchmark.
- **k_neighbors=5** — each synthetic sample is generated by interpolating between a real minority sample and one of its 5 nearest neighbours.
- **Post-SMOTE result:** Balanced 50/50 training set, giving the model equal exposure to both classes during weight updates.

Feature Scaling

StandardScaler (zero mean, unit variance) was fitted on training data and applied to both training and test sets. Scaling was used *only* for Logistic Regression, which is sensitive to feature magnitude. Random Forest and XGBoost are tree-based — they split on rank order, not absolute values — and received the unscaled (but encoded) feature matrix.

7. Model Development

Three-model hierarchy from interpretable to advanced

Three models were trained in a deliberate progression from interpretable baseline to powerful ensemble — each serving a distinct analytical purpose.

7.1 Logistic Regression — Baseline

Purpose: Establish a minimum performance benchmark and provide a fully interpretable linear model whose coefficients can be directly inspected to understand the direction and magnitude of each feature's influence on late delivery probability.

Hyperparameter	Value	Reason
C (regularisation)	1.0	Default — moderate L2 penalty
solver	lbfgs	Efficient for medium datasets
max_iter	1,000	Allows full convergence
class_weight	balanced	Compensates for class imbalance
Input	Scaled data	LR is sensitive to feature magnitude

Results: Accuracy 0.6957, Precision 0.8388, Recall 0.5508, F1 0.6650, ROC-AUC **0.7361**. The high precision (0.84) with low recall (0.55) indicates the model is conservative — it only flags orders as late when very confident, missing many true late deliveries. This precision-recall tradeoff is a known characteristic of linear models on non-linearly separable data.

7.2 Random Forest — Ensemble Tree

Purpose: Capture non-linear feature interactions that Logistic Regression misses. Random Forest trains many independent decision trees on bootstrap samples and averages their predictions, reducing variance through bagging.

Hyperparameter	Value	Reason
n_estimators	200	Strong ensemble without excess compute
max_depth	15	Deep enough for complex patterns, capped to avoid overfit
min_samples_split	10	Prevents leaf nodes on very few samples
min_samples_leaf	5	Regularises tree depth
max_features	sqrt(n)	Standard for classification — limits correlation between trees
class_weight	balanced	Compensates for class imbalance

Hyperparameter	Value	Reason
oob_score	True	Out-of-bag score as a free validation estimate
Input	Raw encoded	Trees do not require scaling

Results: Accuracy 0.7170, Precision 0.8797, Recall 0.5606, F1 0.6848, ROC-AUC **0.7988** — a meaningful improvement over the linear baseline, confirming the presence of non-linear patterns.

7.3 XGBoost — Gradient Boosted Trees

Purpose: The most powerful model in the pipeline. XGBoost uses gradient boosting — each successive tree corrects the residual errors of the previous ensemble — producing consistently state-of-the-art results on tabular data classification tasks.

Hyperparameter	Value	Reason
n_estimators	300	More trees to reduce bias; early stopping via eval_set
max_depth	6	Standard depth for gradient boosting — deeper than 6 risks overfit
learning_rate	0.10	Moderate shrinkage — balances speed vs. accuracy
subsample	0.80	Row subsampling — reduces variance, prevents overfit
colsample_bytree	0.80	Feature subsampling — further variance reduction
min_child_weight	3	Minimum sum of instance weight in a leaf
reg_alpha	0.10	L1 regularisation — feature selection pressure
reg_lambda	1.00	L2 regularisation — weight decay
scale_pos_weight	auto	Computed as #neg / #pos — handles class imbalance
eval_metric	logloss	Log-loss for probabilistic calibration monitoring
Input	Raw encoded	Trees do not require scaling

Results: Accuracy 0.7263, Precision 0.8592, Recall 0.5989, F1 0.7058, ROC-AUC **0.8146**. XGBoost achieved the best balance across all metrics — notably a recall improvement of +4.8 percentage points over Random Forest, meaning it catches more true late deliveries while maintaining high precision.

8. Cross-Validation & Model Selection

Robust evaluation and champion model identification

A single train/test split can yield results that are sensitive to which random samples happened to end up in each partition. Stratified K-Fold cross-validation provides a more reliable performance estimate by training and validating across 5 different partitions of the training data.

Cross-Validation Configuration

Setting	Value	Reason
Folds (k)	5	Standard trade-off between variance and compute time
Strategy	StratifiedKFold	Each fold maintains original class distribution
Metrics	F1 + ROC-AUC	F1 for overall balance; AUC for ranking quality
Training data	SMOTE-resampled train set	Models were evaluated on balanced training data

Cross-Validation Results

Model	F1 Mean	F1 Std	AUC Mean	AUC Std
Logistic Regression	0.6628	±0.0037	0.7553	±0.0033
Random Forest	0.6816	±0.0032	0.8268	±0.0024
XGBoost	0.7018	±0.0035	0.8392	±0.0024

Low standard deviations (≤ 0.004) across all models confirm stable performance — results are not an artefact of a lucky single split. XGBoost's AUC of **0.8392** with a Std of only 0.0024 is particularly encouraging for production deployment.

Final Test Set Comparison

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Logistic Regression	0.6957	0.8388	0.5508	0.6650	0.7361
Random Forest	0.7170	0.8797	0.5606	0.6848	0.7988
XGBoost ★ Champion	0.7263	0.8592	0.5989	0.7058	0.8146

Champion Model: XGBoost — Selected based on highest ROC-AUC (0.8146) and F1 Score (0.7058). The ROC-AUC is the primary selection criterion because it measures the model's ability to *rank* orders by risk — critical for the operations action panel where we want the top-ranked orders to truly be the highest-risk ones, regardless of the exact probability threshold.

9. Risk Probability & Classification

Translating raw probabilities into actionable risk tiers

The final XGBoost model outputs a continuous probability between 0 and 1 for each order. These probabilities were applied to the full 180,519-order dataset to generate the production risk dataset.

Risk Tier Definitions

Risk Tier	Probability Range	Order Count	Percentage	Recommended Action
■ Low Risk	< 0.40	73,712	40.8%	Monitor — standard processing
■ Medium Risk	0.40 – 0.70	47,791	26.5%	Alert — flag for review
■ High Risk	≥ 0.70	59,016	32.7%	Immediate action — escalate now

Threshold Selection Rationale

The thresholds of 0.40 (Medium) and 0.70 (High) were chosen to balance operational utility against alert fatigue:

- **High Risk ≥ 0.70:** ~33% of orders — a manageable queue size for the operations team to handle with immediate escalation. Below this threshold, confidence is insufficient to justify priority overrides.
- **Medium Risk 0.40–0.70:** Orders in a "watch zone" — not certain enough to escalate, but worth monitoring. These appear in daily review queues.
- **Low Risk < 0.40:** Standard processing — no additional intervention. The model is reasonably confident these will arrive on time.
- **Adjustable via Streamlit slider:** The thresholds are not hard-coded — the dashboard's risk threshold slider lets operations managers tune the High Risk cutoff based on their capacity and risk appetite on a given day.

Output Dataset

The full risk-scored dataset (APL_Logistics_with_risk_scores.csv) was saved with two additional columns appended to the original cleaned data: Late_Delivery_Probability (float, 0–1) and Risk_Category (string: Low Risk / Medium Risk / High Risk). This enriched dataset powers all four tabs of the Streamlit dashboard.

10. SHAP Explainability

Making the model transparent and operationally trustworthy

Black-box predictions, however accurate, are insufficient for operational deployment in a logistics context. Operations managers need to understand *why* an order is flagged as high-risk before they can act confidently on the flag. SHAP (SHapley Additive exPlanations) provides this transparency through mathematically rigorous game-theoretic attribution.

What SHAP Measures

For each prediction, SHAP assigns every feature a "Shapley value" — the average marginal contribution of that feature across all possible subsets of features. Positive SHAP values push the prediction toward "Late" (1); negative values push toward "On Time" (0). The **TreeExplainer** variant was used (efficient exact computation for tree-based models like XGBoost), applied to a 3,000-sample subset of the test set.

Top 10 Most Important Features (by Mean |SHAP|)

Rank	Feature	Type	Key Finding
1	tight_schedule_flag	Engineered	Strongest predictor — orders with ≤ 2 scheduled days are drastically riskier
2	Days for shipment (scheduled)	Original	Continuous version of above — shorter = riskier, monotonically
3	Shipping Mode	Original	Mode choice significantly affects risk profile beyond just speed
4	express_shipping_flag	Engineered	Binary version — express modes have different operational dynamics
5	Order Status	Original	PENDING/ON_HOLD statuses highly correlated with delay
6	high_discount_flag	Engineered	Discounted items show elevated delay risk — possible stock/priority issue
7	shipping_mode_risk_score	Engineered	Ordinal risk encoding amplifies signal from Shipping Mode
8	market_risk_score	Engineered	Geographic market delay history is a meaningful predictor
9	Product Price	Original	Price correlates with shipping priority — lower price $\rightarrow$ lower priority
10	Type (payment method)	Original	Payment type correlates with customer risk profile

Key Insights from SHAP Analysis

- **Engineered features dominate the top 10:** 5 of the top 10 features by SHAP importance were created during Feature Engineering, not present in the original dataset. This validates the FE step as critically value-adding.
- **Schedule tightness is the #1 predictor:** Both the binary `tight_schedule_flag` and continuous Days for shipment (scheduled) rank 1st and 2nd. The operational message is clear: when logistics promises extremely short windows, the failure rate spikes.
- **Geography encodes systemic risk:** `market_risk_score` (rank 8) captures that certain geographic corridors — particularly LATAM and Africa — face structural delivery challenges beyond any individual order's control.
- **Beeswarm plot directionality:** The SHAP beeswarm confirms that high values of `tight_schedule_flag`, high `shipping_mode_risk_score`, and PENDING Order Status consistently push predictions toward "Late" (positive SHAP), while Same Day shipping and low `region_risk_score` push toward "On Time" (negative SHAP).
- **Per-order explanations:** The Streamlit Order-Level Prediction tab surfaces these SHAP-derived insights as a "Key Risk Drivers" horizontal bar chart for every individual order prediction, making the model's reasoning transparent to non-technical operations staff.

11. Artifacts & Streamlit Deployment

Saving trained models and launching the interactive dashboard

Saved Artifacts

Artifact	Path	Purpose
logistic_regression.pkl	/content/models/	Baseline model for comparison
random_forest.pkl	/content/models/	RF ensemble model
xgboost_model.pkl	/content/models/	Champion XGBoost model
best_model.pkl	/content/models/	Alias to XGBoost — used by app
scaler.pkl	/content/models/	StandardScaler fitted on train data
label_encoders.pkl	/content/models/	Dict of LabelEncoder per column
feature_info.json	/content/models/	Feature list, mappings, thresholds
APL_Logistics_with_risk_scores.csv	/content/outputs/	Full dataset + probability + tier
model_comparison.csv	/content/outputs/	All model metrics summary

Streamlit Application — Architecture

The Streamlit app (apl_logistics_app.py) was built as a 4-tab multi-page dashboard deployed via ngrok in Google Colab. It loads all saved artifacts once at startup (@st.cache_resource) and the risk-scored dataset once (@st.cache_data), ensuring fast response times on subsequent interactions.

Tab	Module Name	Key Features
Tab 1	Delay Risk Overview	5 KPI cards (total, high/med/low risk counts, avg score) · Risk category pie chart · Probability histogram with threshold lines · Risk by shipping mode & segment
Tab 2	Order-Level Prediction	12 user-input fields across 3 columns · Real-time XGBoost prediction · Risk tier badge with colour-coded probability · Contributing factors horizontal bar chart
Tab 3	Region & Mode Analysis	Regional risk heatmap (horizontal bar) · Stacked bar by mode × tier · Market treemap (size=volume, colour=risk) · Market × Shipping Mode pivot heatmap
Tab 4	Operations Action Panel	Filterable high-risk order table (200 rows) · 4 KPI cards (queue size, revenue at risk, avg score, markets) · CSV download buttons · Prioritisation scatter plot

Sidebar Filters (Global — Affect All Tabs)

Filter	Type	Options
Shipping Mode	Dropdown	All / Standard Class / Second Class / First Class / Same Day
Market / Region	Dropdown	All / LATAM / Europe / Pacific Asia / USCA / Africa
Customer Segment	Dropdown	All / Consumer / Corporate / Home Office
Risk Threshold	Slider (0–1)	Default 0.70 — adjusts "High Risk" cutoff across all views

Deployment in Google Colab

Since Colab does not natively expose ports, **pyngrok** was used to create a secure HTTP tunnel from the Streamlit server (port 8501) to a publicly accessible URL. The tunnel remains active as long as the Colab kernel is running. The public URL was shared with stakeholders for live demonstration.

12. Conclusion & Recommendations

What was achieved and what comes next

What Was Achieved

This project successfully delivered a complete end-to-end machine learning pipeline that transforms APL Logistics' supply chain operations from reactive delay management to predictive risk intelligence. The system can now answer the question — *which orders will be late, before they ship* — with a ROC-AUC of 0.8146 and F1 Score of 0.7058.

Achievement	Outcome
Leakage-free modelling	Identified and removed Delivery Status and real shipping days before training — ensuring genuine predictive validity
Domain-driven FE	Created 10 engineered features; the top predictor (tight_schedule_flag) was engineered, not original
Best-in-class model	XGBoost champion with ROC-AUC 0.8146 and stable 5-fold CV (± 0.0024)
Operational risk tiers	59,016 High Risk orders identified — 32.7% of the dataset — with actionable priority queues
Transparent explainability	SHAP analysis makes the model auditable: schedule tightness and mode risk are the primary drivers
Interactive deployment	4-tab Streamlit dashboard with live filtering, prediction, and CSV export

Operational Recommendations

- **1. Prioritise the High Risk queue daily.** The 32.7% of orders in the High Risk tier represent immediate escalation candidates. Assign dedicated agents to review and intervene on these orders each morning.
- **2. Address tight-schedule ordering patterns.** Since tight_schedule_flag is the #1 predictor, the business should review whether 1–2 day scheduled windows are realistic for current operational capacity — or whether SLA commitments need adjustment.
- **3. Geo-targeted logistics partnerships.** LATAM and Africa consistently show elevated risk scores. Consider dedicated carrier agreements or expanded regional distribution hubs for these markets.
- **4. Integrate predictions into the OMS.** The model's predictions should ideally be scored at order placement time and embedded in the Order Management System so operations staff see risk flags without needing to open a separate dashboard.
- **5. Retrain quarterly.** Logistics patterns shift over time (carrier changes, seasonal demand, new markets). A quarterly retraining cadence on fresh data will maintain model accuracy.

Potential Future Enhancements

- **Date/time features:** Extracting order day-of-week, month, and holiday proximity could capture seasonal patterns not currently modelled.
- **NLP on Product Name:** Certain product categories may have inherent handling complexity — text embeddings of Product Name could surface this.
- **Real-time scoring API:** Wrapping the XGBoost model in a FastAPI service would enable real-time scoring from the OMS at order placement.
- **Threshold optimisation:** The 0.70 High Risk threshold could be optimised using cost-sensitive analysis (e.g., cost of a missed late delivery vs. cost of an unnecessary escalation).
- **Multi-class risk:** Expanding the target to predict specific delivery delay magnitude (1 day late, 2–3 days, 4+ days) would enable graduated response protocols.

Project Impact Summary: By shifting from reactive delay management to predictive risk intelligence, APL Logistics can now flag 59,016 high-risk orders for proactive intervention — *before they ship*. With an ROC-AUC of 0.8146, the XGBoost model correctly ranks delivery risk with over 81% discrimination power, providing operations teams a reliable, explainable, and actionable daily risk intelligence tool.